

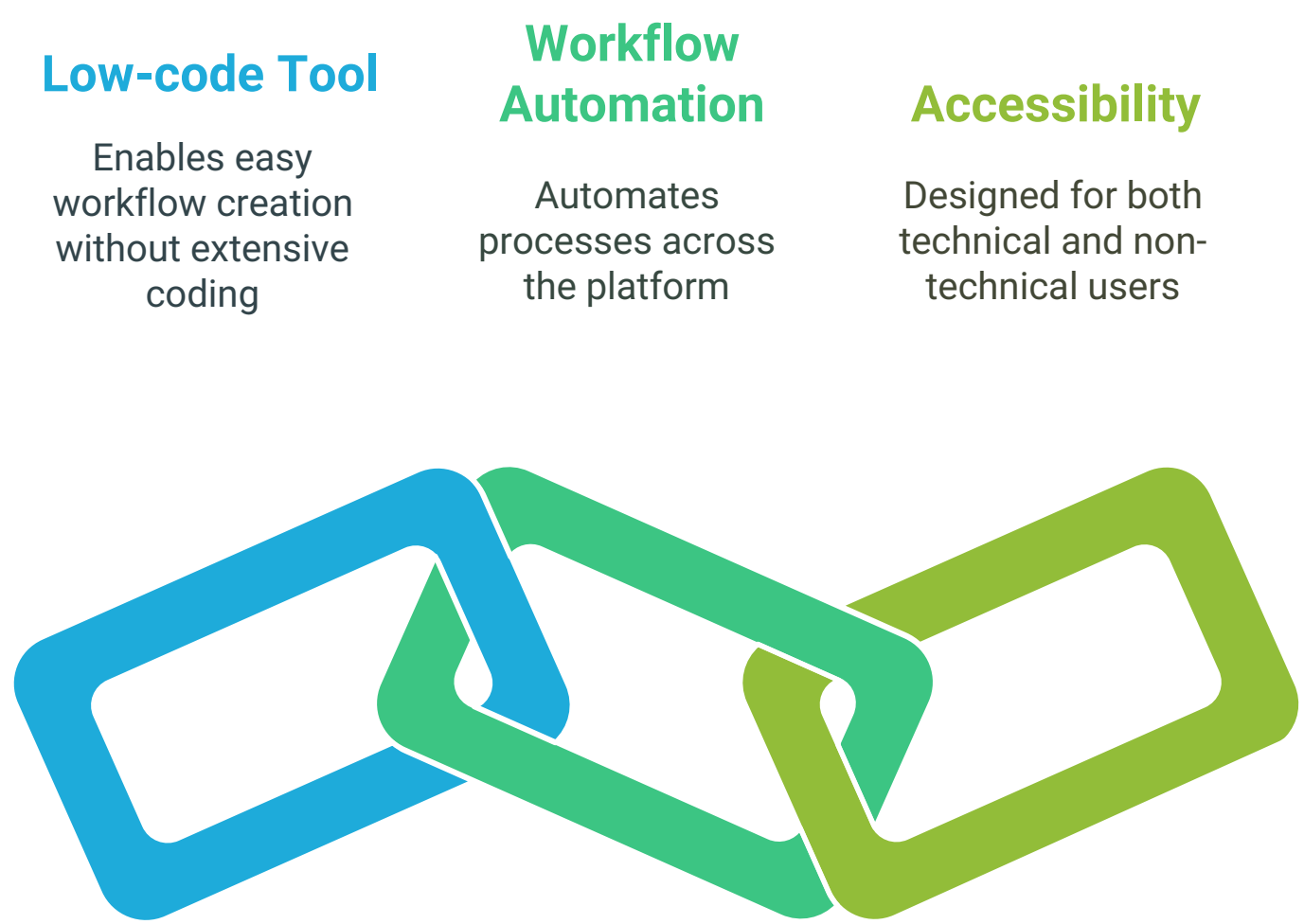
ServiceNow Flow Designer Manual

1. Introduction

Overview of Flow Designer

ServiceNow Flow Designer is a modern, low-code tool used for building and automating workflows across the platform. It enables users to design process flows without requiring extensive scripting, making it accessible to both technical and non-technical users.

Understanding ServiceNow Flow Designer



Key Benefits

- **Low-code Development:** Reduces reliance on scripting and speeds up process automation.
- **Reusability:** Allows the creation of reusable components such as subflows and actions.
- **Improved Maintainability:** Provides a visual interface to manage complex workflows easily.
- **Seamless Integration:** Supports integration with external applications via ServiceNow Spokes.

Low-code Development Benefits



Low-code Development

Reduces reliance on scripting and speeds up process automation.

Allows the creation of reusable components such as subflows and actions.

Reusability



Improved Maintainability

Provides a visual interface to manage complex workflows easily.

Supports integration with external applications via ServiceNow Spokes.

Seamless Integration



2. Core Concepts

Flows

Flows are automated processes in Flow Designer consisting of triggers, actions, and logic to perform business automation.

Actions

Actions are predefined or custom steps within a flow, such as creating a record, sending notifications, or integrating with third-party systems.

Triggers

Triggers determine when a flow starts. Examples include:

- **Record-based triggers:** When a record is created, updated, or deleted.
- **Scheduled triggers:** Running flows at specific intervals.
- **Application events:** Triggered by system events.

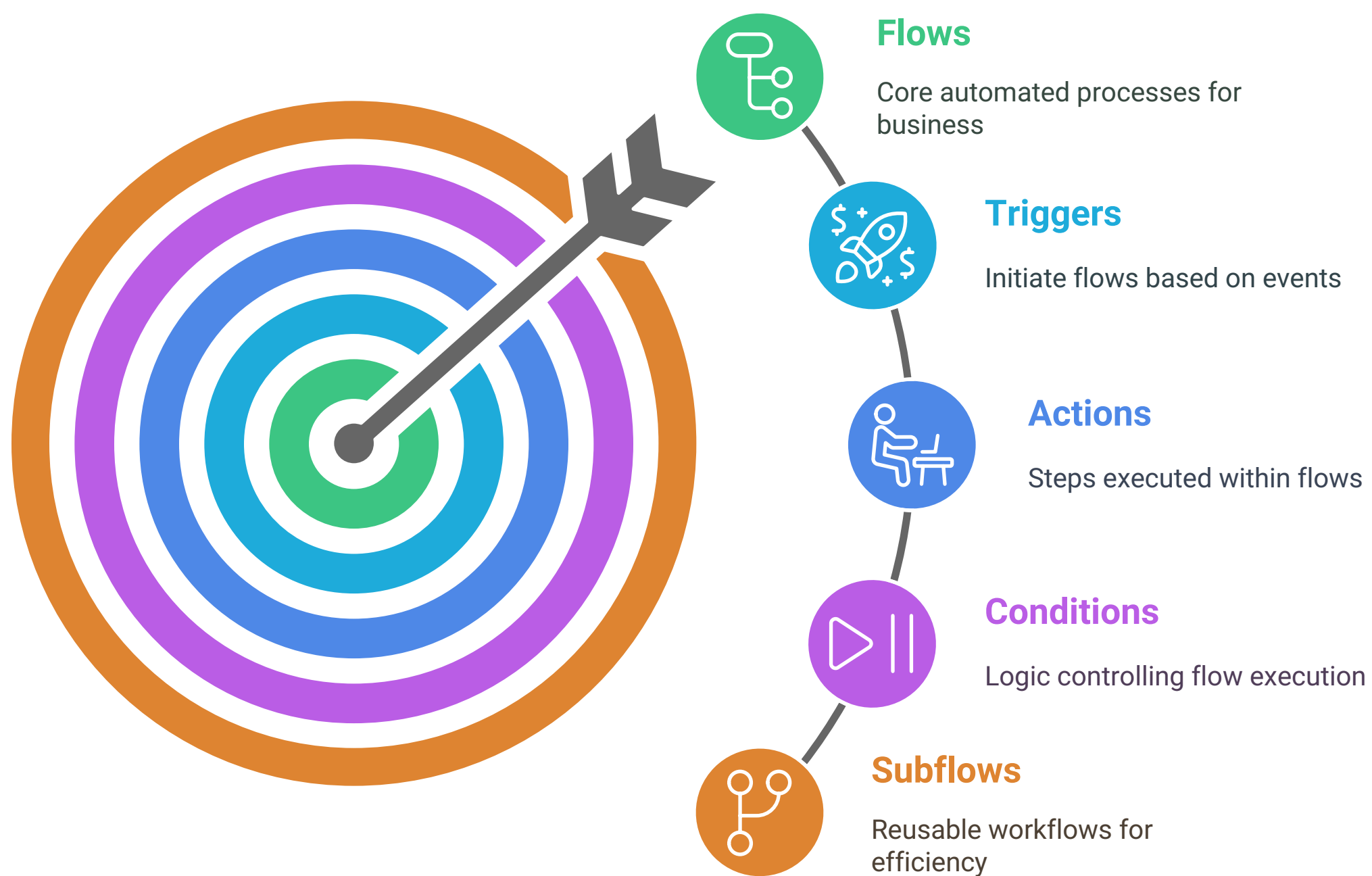
Conditions

Conditions define logic to control the execution path within a flow, such as if-else decisions or switch cases.

Subflows

Reusable smaller workflows that can be called within other flows to standardize processes and reduce redundancy.

Flow Automation Structure



3. Building a Flow

Creating a New Flow

1. Navigate to **Flow Designer** in ServiceNow.
2. Click **Create New Flow**.
3. Provide a **Name** and **Description**.
4. Select the **Trigger Type**.
5. Click **Submit** to start designing the flow.

Defining Triggers

- Select **Record-based triggers** to initiate flows based on table changes.
- Use **Scheduled triggers** to run periodic processes.
- Employ **Application events** for event-driven automation.

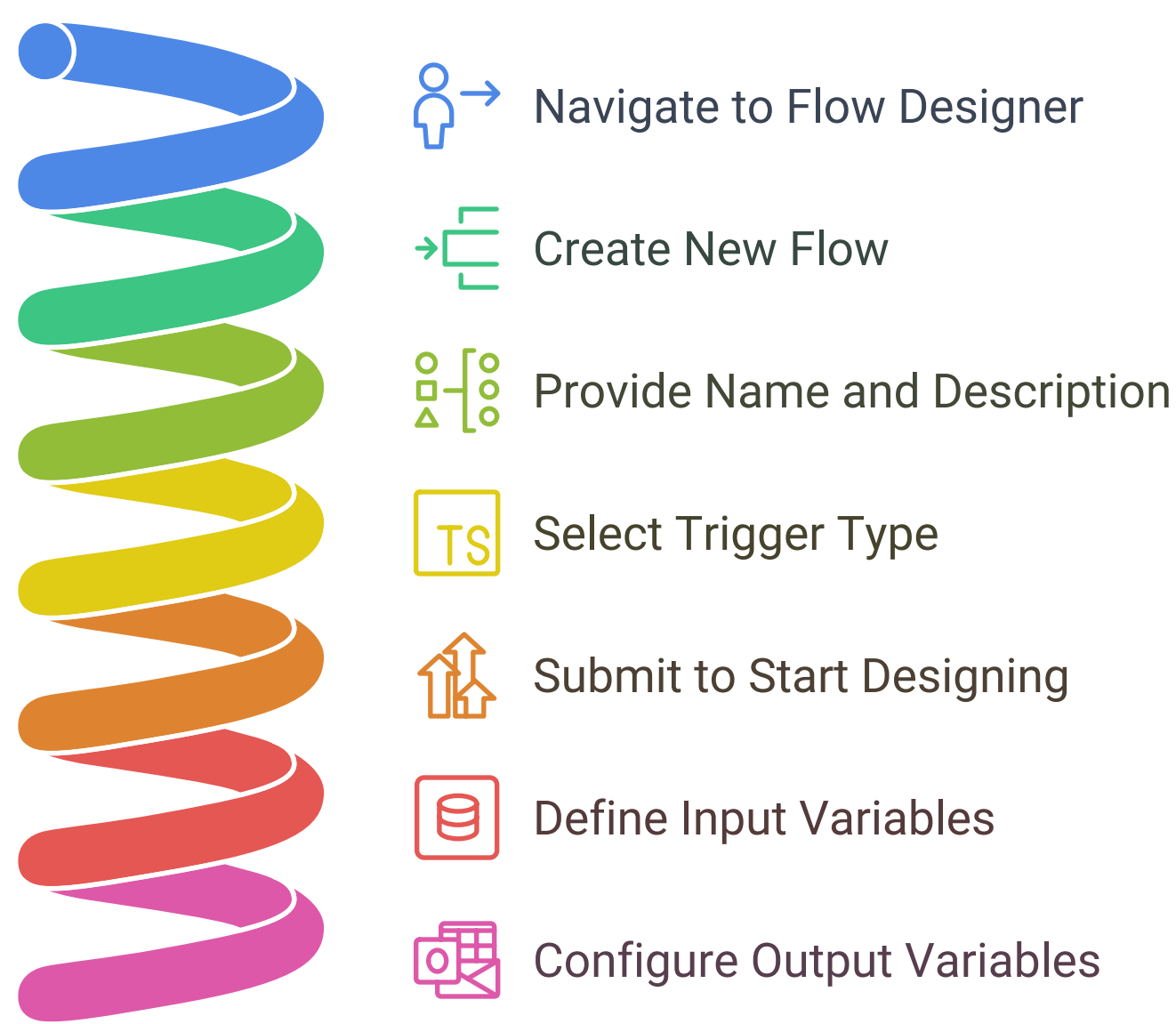
Adding Actions

- Use built-in **Core Actions** such as record creation and approvals.
- Utilize **Spokes** to integrate with external applications.
- Implement **Custom Actions** for tailored automation.

Using Flow Variables

- Define **Input Variables** to pass data into a flow.
- Configure **Output Variables** to return data after execution.

Creating and Configuring a Flow in ServiceNow



4. Subflows

When to Use Subflows

- To standardize repetitive tasks.
- To reduce complexity in large flows.

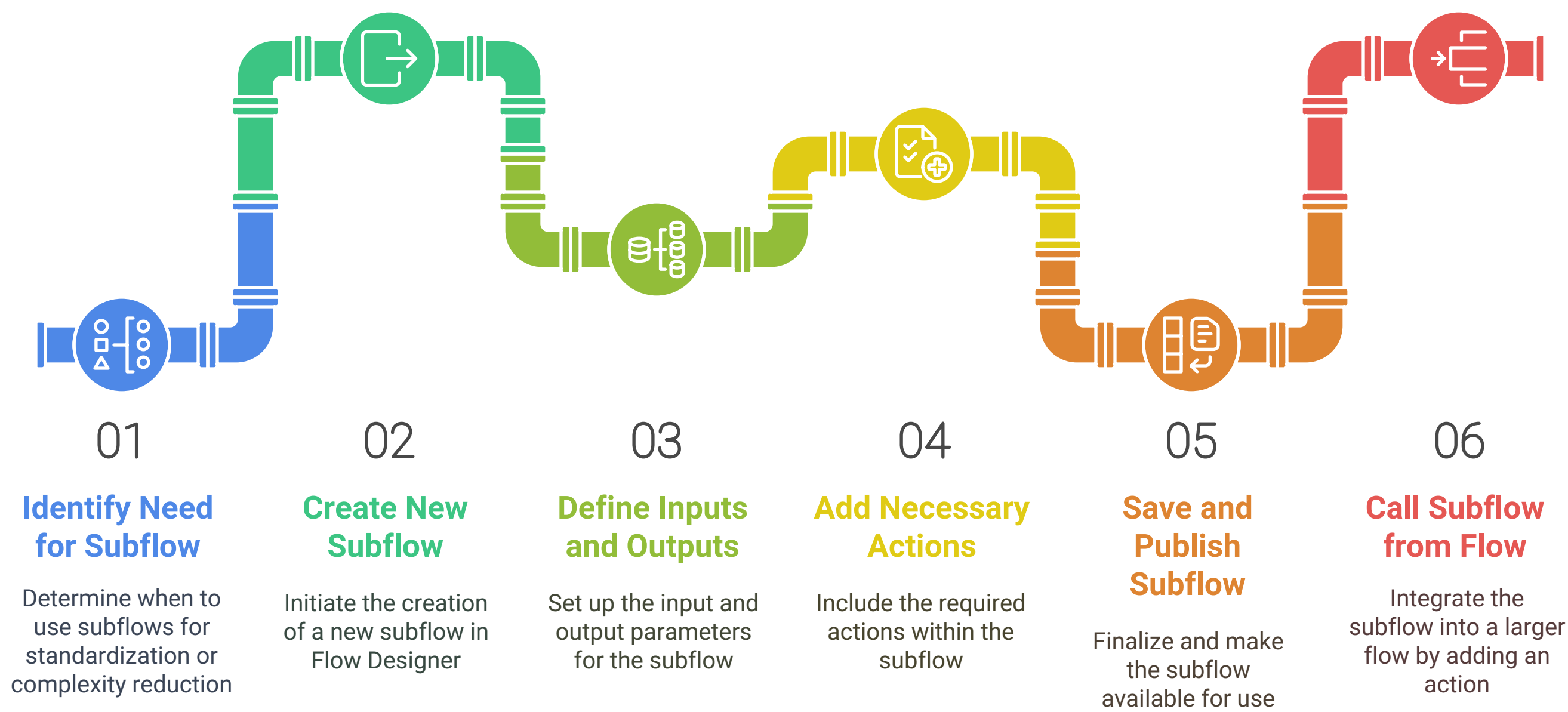
Creating and Configuring Subflows

1. Go to **Flow Designer**.
2. Click **Create New Subflow**.
3. Define **Inputs and Outputs**.
4. Add necessary actions.
5. Save and publish.

Calling a Subflow from a Flow

1. In a flow, add an **Action**.
2. Select **Subflow**.
3. Choose the desired subflow and configure parameters.

Subflow Management in Flow Designer



5. Action Designer (Custom Actions)

Introduction to Action Designer

Custom actions allow developers to extend Flow Designer by creating reusable, scripted actions.

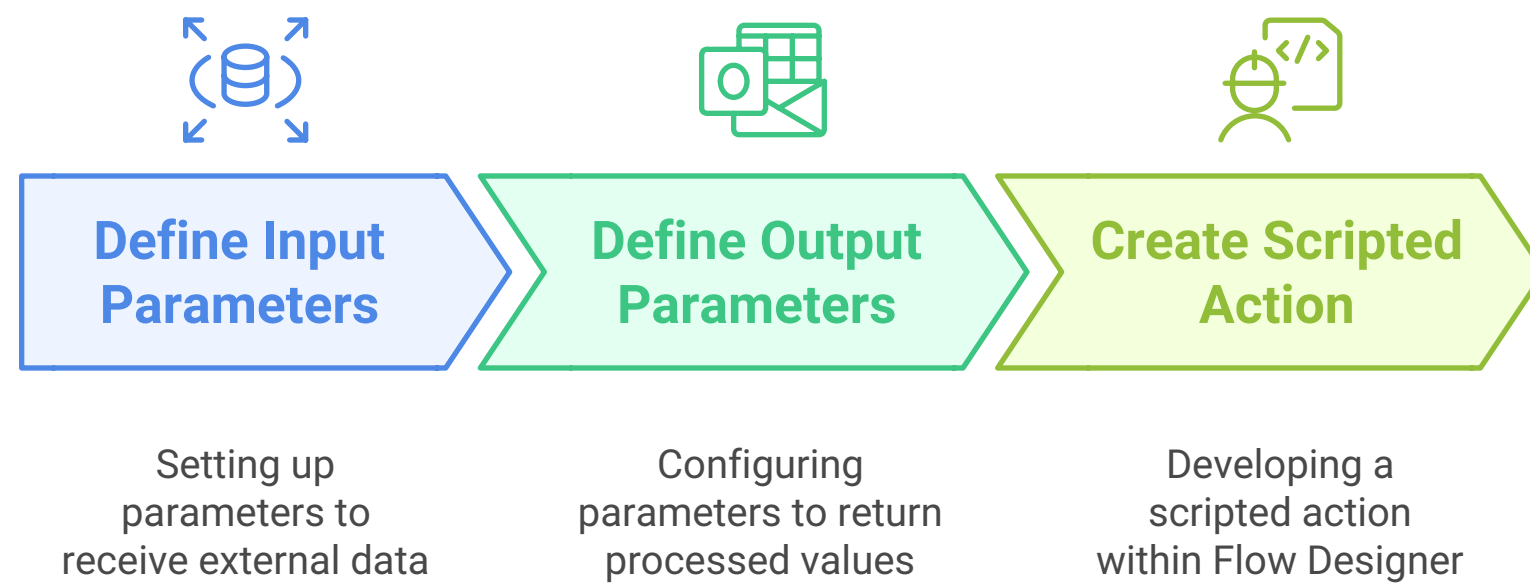
Input/Output Variables

- Define **Input Parameters** to receive external data.
- Use **Output Parameters** to return values after processing.

Scripted Actions

To create a scripted action in Flow Designer:

Creating Scripted Actions in Flow Designer



```
(function execute(inputs, outputs) {  
    var gr = new GlideRecord('incident');  
    gr.initialize();  
    gr.short_description = inputs.short_description;  
    gr.urgency = inputs.urgency;  
    gr.insert();  
    outputs.record_sys_id = gr.sys_id;  
})(inputs, outputs);
```

1. Open **Action Designer**.
2. Click **Create New Action**.
3. Define input and output variables.
4. Write **Script Steps** for execution.
5. Test and save the action.

6. Flow Execution & Troubleshooting

Testing and Debugging Flows

- Use **Test Flow** to validate functionality.
- Enable **Flow Logs** to track execution steps.

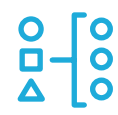
Flow Execution Order

Flows execute in the order defined by their trigger and action sequences.

Handling Errors and Logging

- Implement **Try-Catch Blocks** in scripts.
- Use **Error Handling Actions** to redirect failed executions.

Flow Management Process



Test Flow

Validate functionality of the flow



Enable Flow Logs

Track execution steps for debugging



Define Execution Order

Establish sequence of triggers and actions



Implement Error Handling

Manage errors with try-catch blocks



Example of error handling:

```
try {
  var gr = new GlideRecord('incident');
  gr.get('sys_id', inputs.sys_id);
  outputs.short_description = gr.short_description;
} catch (error) {
  gs.error('Error fetching incident: ' + error.message);
}
```

Monitoring Flow Performance

- Utilize **Flow Designer Analytics** to track execution times.
- Optimize flows by reducing unnecessary actions.

7. Best Practices

Naming Conventions

- Use **descriptive names** for flows and actions.
- Follow **consistent prefixes** (e.g., "HR - Onboarding Flow").

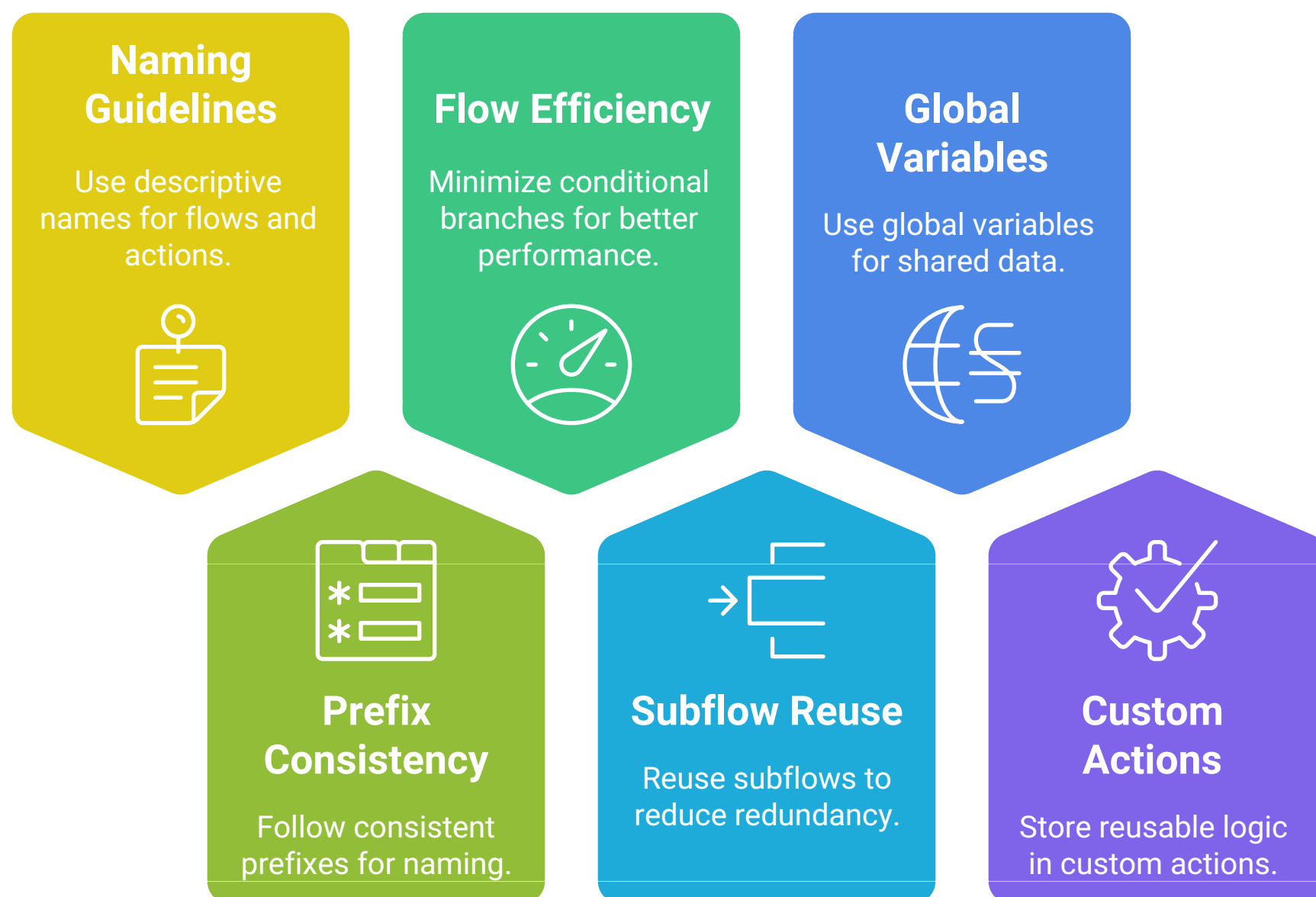
Efficient Flow Design

- Minimize **conditional branches** to improve performance.
- Reuse **subflows** to reduce redundancy.

Reusability and Maintainability

- Use **Global Variables** for shared data.
- Store reusable logic in **Custom Actions**.

Naming and Design Guidelines



8. Use Cases & Examples

Automating Approvals

- Trigger approval processes for HR, finance, or IT service requests.

Integrating with External Systems

- Use **REST API Spokes** to interact with third-party services.

Example of a REST API call in Flow Designer:

```
var request = new sn_ws.RESTMessageV2('ExampleAPI', 'GET');
var response = request.execute();
var responseBody = response.getBody();
outputs.api_response = responseBody;
```

Creating Notifications and Task Assignments

- Automatically send email and mobile notifications based on flow conditions.

9. Advanced Topics

Scripting in Flow Designer

- Use **Script Actions** for custom logic.
- Implement **Data Transforms** using JavaScript.

Using Data Pills Effectively

- Extract and manipulate data from previous steps.
- Reference records dynamically in flows.

Integration with ServiceNow Spokes

- Connect with **JIRA, Slack, Microsoft Teams, and other platforms**.

Performance Optimization

- Use **Batch Actions** where possible.

Enhancing Flow Designer Skills

Performance Optimization

Limit loop iterations for efficiency

Scripting

Use script actions for custom logic

Integration

Connect with platforms like JIRA and Slack

Data Pills

Extract and manipulate data dynamically



10. Security & Governance

Flow Designer Permissions

- Restrict flow editing to **specific user roles**.
- Use **Scoped Applications** for better control.

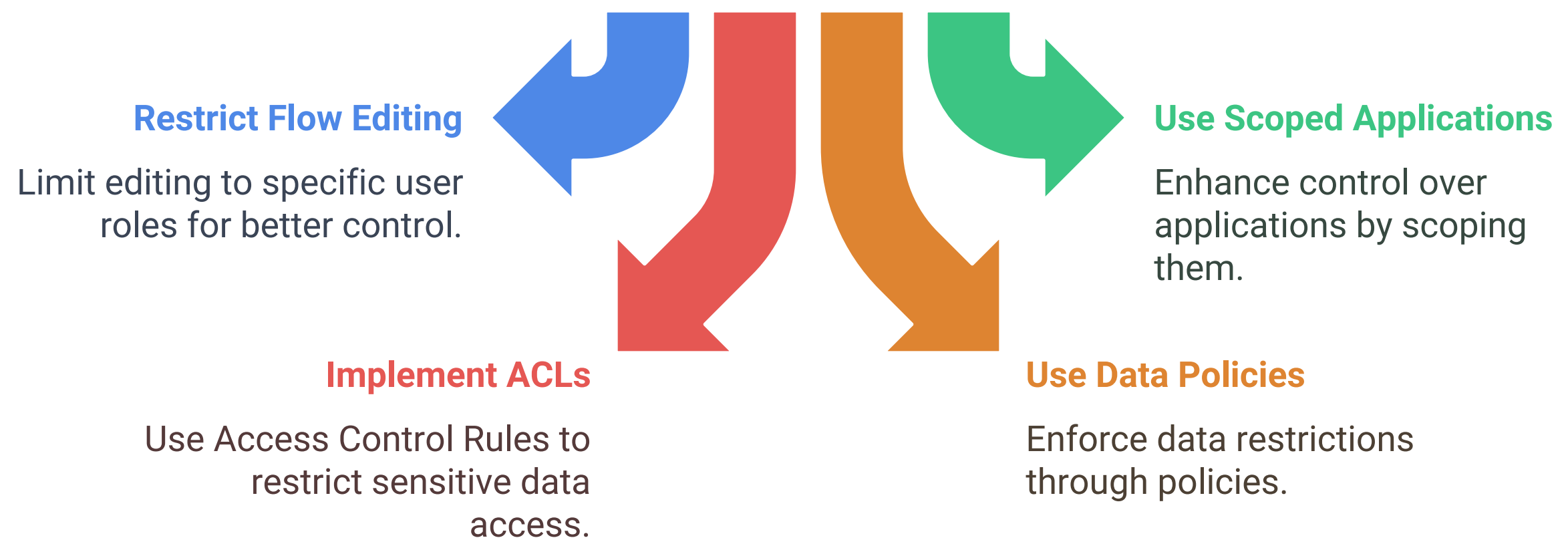
Restricting Access to Sensitive Data

- Implement **Access Control Rules (ACLs)**.
- Use **Data Policies** to enforce restrictions.

Managing Flow Executions at Scale

- Use **Instance Management Tools** to monitor execution limits.
- Optimize **Flow Design** for large-scale automation.

How to manage and secure flow designs and data?



This detailed manual provides an in-depth guide to mastering ServiceNow Flow Designer, ensuring efficient automation and best practices for workflow implementation.